

Day 56: Weekly Revision

Day 56: Weekly Revision — Product and Cart

1. Day Number

Day 56

2. Topic Name

Revision of Days 50–55

Today you will revise:

- Product class
 - Cart class
 - constructor `__init__()`
 - object creation
 - list attributes
 - methods
 - return
 - None
-

3. Connection

In the previous days, you worked with a **Product class** and simple **Cart class methods**.

You learned how to:

```
create Product
  ↓
create Cart
  ↓
add Product to Cart
  ↓
remove Product
  ↓
calculate total
  ↓
search Product
```

Today you will combine the most important parts into one very small program.

4. Revision Summary of Days 50–55

Day 50 — Product Class

You created a class representing a product.

A product can contain attributes such as:

```
name
price
stock
```

Concept:

```
class Product:
    ...
```

Then an object can represent a real product:

```
Product class
  ↓
Laptop object
```

Day 51 — Cart Class

You created a Cart class.

The cart contained a **list attribute**.

Conceptually:

```
Cart
└─ items = []
```

The list stores Product objects.

Day 52 — Add Product

You added a method that accepts a Product object and places it inside the cart.

Main idea:

```
product
  ↓
add_product()
  ↓
cart.items
```

You used:

```
append()
```

Day 53 — Remove Product

You searched through the cart and removed a product when its name matched.

Main concepts:

```
loop
  ↓
check product name
```

```
↓  
match?  
↓  
remove product
```

Day 54 — Calculate Cart Total

You looped through the products and accumulated their prices.

Pattern:

```
total = 0  
  
product 1 → add price  
product 2 → add price  
product 3 → add price  
  
return total
```

This is called an **accumulator pattern**.

Day 55 — Search Product and None

You searched through a list of Product objects.

If the product was found:

```
return product
```

If nothing matched:

```
return None
```

None means:

```
| There is currently no useful object/value to return.
```

5. Important Topics

Class

A class is a blueprint.

Example idea:

```
Product  
├ name  
├ price  
└ stock
```

Object

An object is one actual instance created from a class.

For example:

```
Product class
  ↓
"Keyboard", 1500, 5
```

Constructor

`__init__()` initializes an object's attributes when the object is created.

Basic structure:

```
class Something:
    def __init__(self, value):
        self.value = value
```

List Attribute

An object can contain a list.

For example:

```
Cart object
├── items
│   ├── Product object
│   ├── Product object
│   └── Product object
```

Methods

A method is a function belonging to a class.

Example structure:

```
class Cart:
    def some_method(self):
        ...
```

return

`return` sends a value back to the code that called the method.

Example:

```
def double(number):
    return number * 2
```

Calling:

```
result = double(5)
```

makes:

```
result = 10
```

None

None represents the absence of a value.

Example:

```
result = None
```

It is different from:

```
0  
""  
[]
```

All of those are actual values. None specifically represents **no value/object**.

6. Foundational Notes

self represents the current object

Inside:

```
class Cart:
```

you might have:

```
self.items
```

That means:

| the `items` belonging to this particular `Cart` object.

For example:

```
cart1.items  
cart2.items
```

can be two separate lists.

A list can store objects

A Python list does not have to contain only numbers or strings.

It can contain objects:

```
items
├─ Product("Book", ...)
├─ Product("Mouse", ...)
└─ Product("Keyboard", ...)
```

Therefore:

```
for product in self.items:
```

means:

| Give me each Product object one by one.

Objects have attributes

If product is a Product object, you can access its attributes conceptually using:

```
product.name
product.price
product.stock
```

Methods can modify attributes

For example, an add method can modify:

```
self.items
```

using:

```
append()
```

Methods can also calculate and return values

A total method might:

```
start total at 0
↓
loop through items
↓
add each price
↓
return total
```

It does not necessarily need to print anything itself.

7. Easy Example

Consider something simpler than a shopping cart.

Imagine a Team class that contains player names.

Conceptually:

```
class Team:
    def __init__(self):
        self.players = []

    def add_player(self, player):
        self.players.append(player)
```

Usage:

```
team = Team()

team.add_player("Rahul")
```

Now the object looks conceptually like:

```
team
├──
└── players
    ├── "Rahul"
```

Your Cart works similarly, except instead of storing strings, it stores **Product objects**.

```
cart
├──
└── items
    ├── Product object
```

8. Revision Problem Statement

Create a very simple shopping cart program.

Requirements

Create:

- one Product class
- one Cart class
- one Product object
- one Cart object

The Product should have at least:

```
name
price
```

The Cart should have:

```
items = []
```

Add a method that allows a Product object to be added to the cart.

Also add a method that calculates and **returns the total price** of all products in the cart.

Then:

```
Create one Product
      ↓
Create one Cart
      ↓
Add Product to Cart
      ↓
Calculate cart total
      ↓
Print total
```

Keep the program very easy.

9. Concepts Used

You will practice:

1. Classes
2. Objects
3. `__init__()`
4. `self`
5. Object attributes
6. List attributes
7. Methods
8. Method parameters
9. `append()`
10. `for` loop
11. Accumulator variable
12. `return`
13. Object method calls

You do **not** need complicated validation for today's problem.

10. Thought Process

Before writing Python, think about the objects.

Step 1: What represents a product?

A Product object.

It should know its:

```
name
price
```

Step 2: What represents the shopping cart?

A Cart object.

It should contain:

```
items
```

Initially:

```
items = []
```

Step 3: How will a product enter the cart?

Create a method such as:

```
add_product(product)
```

That method should put the Product object into:

```
self.items
```

Step 4: How do we calculate the total?

Start:

```
total = 0
```

Loop through:

```
self.items
```

For every Product:

```
total = total + product.price
```

Finally:

```
return total
```

Step 5: Who prints the total?

The method should **return** the value.

Outside the class, you can print the returned value.

Think of the difference:

```
calculate_total()  
  ↓  
return number
```

↓
print returned number

11. Pseudocode

```
CREATE Product class

    CREATE constructor with name and price

        STORE name
        STORE price

CREATE Cart class

    CREATE constructor

        CREATE empty items list

    CREATE add_product method receiving product

        ADD product to items list

    CREATE calculate_total method

        SET total to 0

        FOR every product in items

            ADD product price to total

        RETURN total

CREATE one Product object

CREATE one Cart object

CALL cart's add_product method
PASS Product object

CALL cart's calculate_total method

PRINT returned total
```

Notice the object relationship:

```
Product object
|
| add_product()
```



12. Suggested Solving Approach

Use an **OOP-based approach**.

A good structure is:



Then outside the classes:

1. Create Product
2. Create Cart
3. Add Product
4. Calculate total
5. Print total

Try not to add remove/search functionality today. The goal is revision, not complexity.

13. Easy Edge Cases

Edge Case 1: Empty Cart

Suppose:

```
items = []
```

The total should naturally be:

```
0
```

Why?

Because:

```
total starts at 0  
  
no products exist  
↓  
loop runs zero times  
↓  
total remains 0
```

So your accumulator pattern already handles this case nicely.

Edge Case 2: Product Price Is 0

Example:

```
Product:  
name = "Free Sample"  
price = 0
```

After adding it:

```
total = 0 + 0
```

Result:

```
0
```

This is valid. A price of 0 should not break the program.

14. Common Mistakes to Avoid

Mistake 1: Forgetting self

Wrong idea:

```
items = []
```

when you actually want the list to belong to the Cart object.

Remember the idea:

```
self.items
```

Mistake 2: Passing the product name instead of Product object

Your cart should store:

```
Product object
```

not merely:

```
"Keyboard"
```

because later you need things like:

```
product.price
```

Mistake 3: Forgetting to initialize total

Before the loop, you need an accumulator conceptually like:

```
total = 0
```

Mistake 4: Replacing instead of adding

Be careful with the difference:

```
total = product.price
```

versus:

```
total = total + product.price
```

The second pattern accumulates prices.

Mistake 5: Printing instead of returning

If `calculate_total()` is supposed to **return** the total, do not make printing its main job.

Think:

```
method → calculates → returns  
caller → receives → prints
```

Mistake 6: Creating the list as a shared class attribute

For now, the safest beginner pattern is to initialize the cart's list inside its constructor:

```
Cart created  
  ↓  
__init__()  
  ↓  
new empty items list
```

15. Quick Self-Check Questions

1. What is the difference between Product and a Product object?

Think about:

blueprint vs actual instance

2. Why does the Cart need an items list?

Think about what happens when you want to store multiple Product objects.

3. What does append() do in add_product()?

Does it create a new list, or add something to the existing list?

4. Why should calculate_total() start with total = 0?

Think about the accumulator pattern.

5. What is the difference between return total and print(total)?

Think about:

return → gives value back
print → displays value

16. Hint Only

Build the solution in this order:

```
Product
  ↓
constructor
  ↓
name + price

Cart
  ↓
constructor
  ↓
empty items list
  ↓
add_product()
  ↓
calculate_total()
```

For your total method, remember this pattern:

```
START with 0

FOR each product
  ADD its price
```

```
RETURN final value
```

And when adding the product, remember that you are adding the **whole Product object**:

```
Cart
└─ items
    └─ Product object
        ├── name
        └─ price
```

That is enough to solve **Day 56** without needing remove or search functionality.